

FORMATION EN CREATION DE SITE INTERNET DYNAMIQUE

*FORMATEUR : Samuel KOTEI-NIKOI,
Ingénieur de Conception en Informatique,
Contacts : +225 05 93 91 86 / +225 40 95 69 76*

PARTIE 2 : TRANSMETTRE DES DONNEES DE PAGE EN PAGE

- I. Transmettre des données avec l'URL
- II. Transmettre des données avec les formulaires
- III. Variables superglobales, sessions et cookies

I : Transmettre des données avec l'URL

1. Envoyer des paramètres dans l'URL

Former une URL pour envoyer des paramètres

Imaginons que votre site s'appelle monsite.com et que vous avez une page PHP intitulée bonjour.php. Pour accéder à cette page, vous devez aller à l'URL suivante :

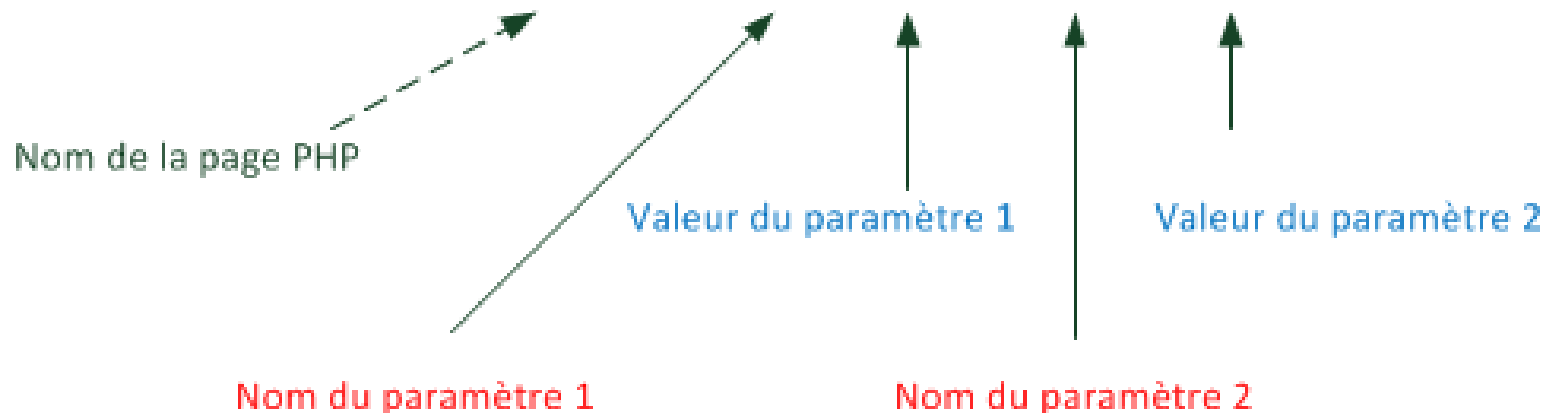
<http://www.monsite.com/bonjour.php>

Jusque-là, rien de bien nouveau. Ce que nous allons apprendre à faire, c'est d'**envoyer** des informations à la page bonjour.php. Pour cela, on va ajouter des informations à la fin de l'URL, comme ceci :

<http://www.monsite.com/bonjour.php?nom=Dupont&prenom=Jean>

Ce que vous voyez après le point d'interrogation, ce sont des **paramètres** que l'on envoie à la page PHP. Celle-ci peut récupérer ces informations dans des variables. Voyez sur la figure suivante comment on peut découper cette URL.

`http://www.monsite.com/bonjour.php?nom=Dupont&prenom=Jean`



Le point d'interrogation sépare le nom de la page PHP des paramètres. Ensuite, ces derniers s'enchaînent selon la forme **nom=valeur** et sont séparés les uns des autres par le symbole **&**.

Créer un lien avec des paramètres

Maintenant que nous savons cela, nous pouvons créer des liens en HTML qui transmettent des paramètres d'une page vers une autre.

Imaginons que vous avez deux fichiers sur votre site :

index.php (l'accueil) ;

bonjour.php.

Nous voulons faire un lien de index.php qui mène à bonjour.php et qui lui transmet des informations dans l'URL, comme le schématise la figure suivante.



Pour cela, ouvrez index.php (puisque c'est lui qui contiendra le lien) et insérez-y par exemple le code suivant :

```
<a href="bonjour.php?nom=Dupont&prenom=Jean">Dis-moi bonjour !</a>
```

Comme vous le voyez, le & dans le code a été remplacé par & dans le lien. Ça n'a rien à voir avec PHP : simplement, en HTML, on demande à ce que les & soient écrits & dans le code source. Si vous ne le faites pas, le code ne passera pas la validation W3C.

Ce lien appelle la page `bonjour.php` et lui envoie deux paramètres :

- nom : Dupont ;
- prenom : Jean.

Vous avez sûrement deviné ce qu'on essaie de faire ici : on appelle une page `bonjour.php` qui va dire « Bonjour » à la personne dont le nom et le prénom ont été envoyés en paramètres. Comment faire dans la page `bonjour.php` pour récupérer ces informations ? C'est ce que nous allons voir maintenant.

2. Récupérer les paramètres en PHP

Nous savons maintenant comment former des liens pour envoyer des paramètres vers une autre page. Nous avons pour cela ajouté des paramètres à la fin de l'URL.

Intéressons-nous maintenant à la page qui réceptionne les paramètres. Dans notre exemple, il s'agit de la page `bonjour.php`. Celle-ci va automatiquement créer un array au nom un peu spécial : `$_GET`. Il s'agit d'un array associatif dont les clés correspondent aux noms des paramètres envoyés en URL.

Reprenons notre exemple pour mieux voir comment cela fonctionne. Nous avons fait un lien vers **`bonjour.php?nom=Dupont&prenom=Jean`**, cela signifie que nous aurons accès aux variables suivantes :

Nom	Valeur
<code>\$_GET['nom']</code>	Dupont
<code>\$_GET['prenom']</code>	Jean

On peut donc récupérer ces informations, les traiter, les afficher, bref faire ce que l'on veut avec. Pour commencer, essayons tout simplement de les afficher :

```
<p>Bonjour <?php echo $_GET['prenom']; ?> !</p>
```

```
<p>Bonjour <?php echo $_GET['prenom'] . ' ' . $_GET['nom']; ?> !</p>
```

I : Transmettre des données avec les formulaires

1. Créer la base du formulaire

En HTML, pour insérer un formulaire, on se sert de la balise `<form>`. On l'utilise de la manière suivante :

```
<form method="post" action="cible.php">
```

```
<p>
```

On insèrera ici les éléments de notre formulaire.

```
</p>
```

```
</form>
```

On écrira le contenu de notre formulaire entre les balises `<form>` et `</form>`.

Il y a deux attributs très importants à connaître pour la balise `<form>` : la méthode (method) et la cible (action).

La méthode

Il existe plusieurs moyens d'envoyer les données du formulaire (plusieurs « méthodes »). Vous pouvez en employer deux.

- **get** : les données transiteront par l'URL comme on l'a appris précédemment. On pourra les récupérer grâce à l'array `$_GET`. Cette méthode est assez peu utilisée car on ne peut pas envoyer beaucoup d'informations dans l'URL
- **post** : les données ne transiteront pas par l'URL, l'utilisateur ne les verra donc pas passer dans la barre d'adresse. Cette méthode permet d'envoyer autant de données que l'on veut, ce qui fait qu'on la privilégie le plus souvent.

La cible

L'attribut **action** sert à définir la page appelée par le formulaire. C'est cette page qui recevra les données du formulaire et qui sera chargée de les traiter.

2. Les éléments du formulaire

Les petites zones de texte : `<input type="text" />`

Pour les mots de passe, vous pouvez utiliser `type="password"`, ce qui aura pour effet de cacher le texte entré par le visiteur.

Il y a deux attributs à connaître que l'on peut ajouter à cette balise.

- **name** (obligatoire) : c'est le nom de la zone de texte. Choisissez-le bien, car c'est lui qui va produire une variable. Par exemple :
`<input type="text" name="pseudo" />`.
- **value** (facultatif) : c'est ce que contient la zone de texte au départ. Par défaut, la zone de texte est vide mais il peut être pratique de pré-remplir le champ. Exemple :
`<input type="text" name="pseudo" value="Matrix" />`.

Le texte que le visiteur aura entré sera disponible dans `cible.php` sous la forme d'une variable appelée `$_POST['pseudo']`

Les grandes zones de texte

La grande zone de texte (on l'appelle aussi « zone de saisie multiligne »)

On peut y écrire autant de lignes que l'on veut. C'est plus adapté si le visiteur doit écrire un long message, par exemple.

```
<textarea name="message" rows="8" cols="45">
```

Votre message ici.

```
</textarea>
```

La liste déroulante

```
<select name="choix">
```

```
<option value="choix1">Choix 1</option>
```

```
<option value="choix2">Choix 2</option>
```

```
<option value="choix3">Choix 3</option>
```

```
<option value="choix4">Choix 4</option>
```

```
</select>
```

Les cases à cocher

```
<input type="checkbox" name="case" id="case" /> <label for="case">Ma case à  
cocher</label>
```

Là encore, on donne un nom à la case à cocher via l'attribut name (ici : « case »). Ce nom va générer une variable dans la page cible, par exemple `$_POST['case']`.

Si la case a été cochée, alors `$_POST['case']` aura pour valeur « on ».

Si elle n'a pas été cochée, alors `$_POST['case']` n'existera pas. Vous pouvez faire un test avec `isset($_POST['case'])` pour vérifier si la case a été cochée ou non.

Si vous voulez que la case soit cochée par défaut, il faudra lui rajouter l'attribut **`checked="checked"`**.

```
<input type="checkbox" name="case" checked="checked" />
```

Les boutons d'option

Les boutons d'option fonctionnent par groupes de deux minimum.

Exemple :

Aimez-vous les frites ?

```
<input type="radio" name="frites" value="oui" id="oui" checked="checked" /> <label  
for="oui">Oui</label>
```

```
<input type="radio" name="frites" value="non" id="non" /> <label for="non">Non</label>
```

Comme vous pouvez le voir, les deux boutons d'option ont le même nom (« frites »). C'est très important, car ils fonctionnent par groupes : tous les boutons d'option d'un même groupe doivent avoir le même nom. Cela permet au navigateur de savoir lesquels désactiver quand on active un autre bouton du groupe.

Pour pré-cocher l'un de ces boutons, faites comme pour les cases à cocher : rajoutez un attribut `checked="checked"`. Ici, comme vous pouvez le voir, « Oui » est sélectionné par défaut.

Dans la page cible, une variable `$_POST['frites']` sera créée. Elle aura la valeur du bouton d'option choisi par le visiteur, issue de l'attribut `value`. Si on aime les frites, alors on aura `$_POST['frites'] = 'oui'`.

Il faut bien penser à renseigner l'attribut `value` du bouton d'option car c'est lui qui va déterminer la valeur de la variable.

Les champs cachés

Les champs cachés constituent un type de champ à part. En quoi ça consiste ? C'est un code dans votre formulaire qui n'apparaîtra pas aux yeux du visiteur, mais qui va quand même créer une variable avec une valeur. On peut s'en servir pour transmettre des informations fixes.

Exemple :

```
<input type="hidden" name="pseudo" value="Mateo21" />
```

3. L'envoi de fichiers

L'envoi de fichier se passe en deux temps.

- On remplit un formulaire en indiquant le fichier à envoyer. Une simple page HTML suffit pour créer le formulaire.
- PHP réceptionne les données du formulaire et, s'il y a des fichiers dedans, il les « enregistre » dans un des dossiers du serveur.

Le formulaire d'envoi de fichier

Dès l'instant où votre formulaire propose aux visiteurs d'envoyer un fichier, il faut ajouter l'attribut **enctype="multipart/form-data"** à la balise <form>.

Exemple :

```
<form action="cible_envoi.php" method="post" enctype="multipart/form-data">  
  <p>Formulaire d'envoi de fichier</p>  
</form>
```

Grâce à **enctype**, le navigateur du visiteur sait qu'il s'apprête à envoyer des fichiers. Maintenant que c'est fait, nous pouvons ajouter à l'intérieur du formulaire une balise permettant d'envoyer un fichier. C'est une balise très simple de type **<input type="file" />**. Il faut penser comme toujours à donner un nom à ce champ de formulaire (grâce à l'attribut name) pour que PHP puisse reconnaître le champ par la suite.

```
<form action="cible_envoi.php" method="post" enctype="multipart/form-data">
  <p>
    Formulaire d'envoi de fichier :<br />
    <input type="file" name="monfichier" /><br />
    <input type="submit" value="Envoyer le fichier" />
  </p>
</form>
```

Le traitement de l'envoi en PHP

Comme vous avez dû le remarquer, le formulaire pointe vers une page PHP qui s'appelle `cible_envoi.php`. Le visiteur sera donc redirigé sur cette page après l'envoi du formulaire. C'est maintenant que ça devient important. Il faut que l'on écrive le code de la page `cible_envoi.php` pour traiter l'envoi du fichier.

Au moment où la page PHP s'exécute, le fichier a été envoyé sur le serveur mais il est stocké dans un **dossier temporaire**.

C'est à vous de décider si vous acceptez définitivement le fichier ou non. Vous pouvez par exemple vérifier si le fichier a la bonne extension (si vous demandiez une image et qu'on vous envoie un « .txt », vous devrez refuser le fichier).

Si le fichier est bon, vous l'accepterez grâce à la fonction **`move_uploaded_file`**, et ce, d'une manière définitive.

Pour chaque fichier envoyé, une variable **`$_FILES['nom_du_champ']`** est créée. Dans notre cas, la variable s'appellera **`$_FILES['monfichier']`**.

Cette variable est un tableau qui contient plusieurs informations sur le fichier :

Variable	Signification
<code>\$_FILES['monfichier']['name']</code>	Contient le nom du fichier envoyé par le visiteur.
<code>\$_FILES['monfichier']['type']</code>	Indique le type du fichier envoyé. Si c'est une image gif par exemple, le type sera image/gif.
<code>\$_FILES['monfichier']['size']</code>	Indique la taille du fichier envoyé. Attention : cette taille est en octets. Il faut environ 1 000 octets pour faire 1 Ko, et 1 000 000 d'octets pour faire 1 Mo. Attention : la taille de l'envoi est limitée par PHP. Par défaut, impossible d'uploader des fichiers de plus de 8 Mo.
<code>\$_FILES['monfichier']['tmp_name']</code>	Juste après l'envoi, le fichier est placé dans un répertoire temporaire sur le serveur en attendant que votre script PHP décide si oui ou non il accepte de le stocker pour de bon. Cette variable contient l'emplacement temporaire du fichier (c'est PHP qui gère ça).
<code>\$_FILES['monfichier']['error']</code>	Contient un code d'erreur permettant de savoir si l'envoi s'est bien effectué ou s'il y a eu un problème et si oui, lequel. La variable vaut 0 s'il n'y a pas eu d'erreur.

Si vous avez mis un second champ d'envoi de fichier dans votre formulaire, il y aura une seconde variable `$_FILES['nom_de_votre_autre_champ']` découpée de la même manière que le tableau qu'on vient de voir ici.

`$_FILES['nom_de_votre_autre_champ']['size']` contiendra donc la taille du second fichier, et ainsi de suite.

Proposons de faire les vérifications suivantes pour décider si l'on accepte le fichier ou non.

- Vérifier tout d'abord si le visiteur a bien envoyé un fichier (en testant la variable `$_FILES['monfichier']` avec `isset()`) et s'il n'y a pas eu d'erreur d'envoi (grâce à `$_FILES['monfichier']['error']`).
- Vérifier si la taille du fichier ne dépasse pas 1 Mo par exemple (environ 1 000 000 d'octets) grâce à `$_FILES['monfichier']['size']`.
- Vérifier si l'extension du fichier est autorisée (il faut interdire à tout prix que les gens puissent envoyer des fichiers PHP, sinon ils pourraient exécuter des scripts sur votre serveur). Dans notre cas, nous autoriserons seulement les images (fichiers .png, .jpg, .jpeg et .gif).

Nous analyserons pour cela la variable `$_FILES['monfichier']['name']`.

Nous allons donc faire une série de tests dans notre page `cible_envoi.php`.

```
<?php
// Testons si le fichier a bien été envoyé et s'il n'y a pas d'erreur
if (isset($_FILES['monfichier']) AND $_FILES['monfichier']['error'] == 0)
{
    // Testons si le fichier n'est pas trop gros
    if ($_FILES['monfichier']['size'] <= 1000000)
    {
        // Testons si l'extension est autorisée
        $infosfichier = pathinfo($_FILES['monfichier']['name']);
        $extension_upload = $infosfichier['extension'];
        $extensions_autorisees = array('jpg', 'jpeg', 'gif', 'png');
        if (in_array($extension_upload, $extensions_autorisees))
        {
            // On peut valider le fichier et le stocker définitivement
            move_uploaded_file($_FILES['monfichier']['tmp_name'], 'uploads/' .
basename($_FILES['monfichier']['name']));
            echo "L'envoi a bien été effectué !";
        }
    }
}
?>
```


La fonction `pathinfo` renvoie un array contenant entre autres l'extension du fichier dans `$infosfichier['extension']`. On stocke ça dans une variable `$extension_upload`. Une fois l'extension récupérée, on peut la comparer à un tableau d'extensions autorisées (un array) et vérifier si l'extension récupérée fait bien partie des extensions autorisées à l'aide de la fonction `in_array()`.

Si tout est bon, on accepte le fichier en appelant `move_uploaded_file()`.

Cette fonction prend deux paramètres :

- le nom temporaire du fichier (on l'a avec `$_FILES['monfichier']['tmp_name']`) ;
- le chemin qui est le nom sous lequel sera stocké le fichier de façon définitive. On peut utiliser le nom d'origine du fichier `$_FILES['monfichier']['name']` ou générer un nom au hasard.

On peut se proposer par exemple de placer le fichier dans un sous-dossier « uploads ».

On gardera le même nom de fichier que celui d'origine.

Comme `$_FILES['monfichier']['name']` contient le chemin entier vers le fichier d'origine (C:\dossier\fichier.png par exemple), il nous faudra extraire le nom du fichier. On peut utiliser pour cela la fonction **basename** qui renverra juste « fichier.png ».

III : Variables superglobales, sessions et cookies

Vous avez probablement remarqué que les arrays `$_GET` et `$_POST` sont des variables un peu particulières : leur nom est écrit en majuscules et commence par un *underscore* (le trait de soulignement), mais surtout ces variables sont générées automatiquement par PHP. Ce sont ce qu'on appelle des **variables superglobales**. Il existe d'autres types de variables superglobales que nous allons découvrir dans ce chapitre. Parmi elles, certaines permettent de stocker des informations pendant la durée d'une visite, c'est le principe des sessions, mais aussi de stocker des informations sur l'ordinateur de vos visiteurs pendant plusieurs mois, c'est le principe des cookies.

1. Les variables superglobales

Les variables superglobales sont des variables un peu particulières pour trois raisons :

- elles sont écrites en majuscules et commencent toutes, à une exception près, par un underscore (`_`). `$_GET` et `$_POST` en sont des exemples que vous connaissez ;
- les superglobales sont des array car elles contiennent généralement de nombreuses informations ;
- enfin, ces variables sont automatiquement créées par PHP à chaque fois qu'une page est chargée. Elles existent donc sur toutes les pages et sont accessibles partout : au milieu de votre code, au début, dans les fonctions, etc.

Pour afficher le contenu d'une superglobale et voir ce qu'elle contient, le plus simple est d'utiliser la fonction **`print_r`**, puisqu'il s'agit d'un array. Exemple :

```
<pre>
    <?php
        print_r($_GET);
    ?>
</pre>
```

Passons en revue les principales variables superglobales existantes. Nous ne les utiliserons pas toutes, mais nous aurons fait un petit tour d'horizon pour pouvoir nous concentrer ensuite sur les plus utiles d'entre elles.

- **\$_SERVER** : ce sont des valeurs renvoyées par le serveur. Elles sont nombreuses et quelques-unes d'entre elles peuvent nous être d'une grande utilité. Par exemple **\$_SERVER['REMOTE_ADDR']**. Elle nous donne l'adresse IP du client qui a demandé à voir la page, ce qui peut être utile pour l'identifier.
- **\$_ENV** : ce sont des variables d'environnement toujours données par le serveur. C'est le plus souvent sous des serveurs Linux que l'on retrouve des informations dans cette superglobale. Généralement, on ne trouvera rien de bien utile là-dedans pour notre site web.
- **\$_SESSION** : on y retrouve les variables de session. Ce sont des variables qui restent stockées sur le serveur le temps de la présence d'un visiteur. Nous allons apprendre à nous en servir dans ce chapitre.
- **\$_COOKIE** : contient les valeurs des cookies enregistrés sur l'ordinateur du visiteur. Cela nous permet de stocker des informations sur l'ordinateur du visiteur pendant plusieurs mois, pour se souvenir de son nom par exemple.

- **`$_GET`** : vous la connaissez, elle contient les données envoyées en paramètres dans l'URL.
- **`$_POST`** : de même, c'est une variable que vous connaissez et qui contient les informations qui viennent d'être envoyées par un formulaire.
- **`$_FILES`** : elle contient la liste des fichiers qui ont été envoyés via le formulaire précédent.

Vous connaissez déjà une bonne partie de ces variables superglobales, comme vous pouvez le constater. Nous allons étudier plus en détail les sessions et les cookies. Avec ça nous aurons fait le tour des principaux moyens de transmettre des variables de page en page !

2. Les sessions

Les sessions constituent un moyen de conserver des variables sur toutes les pages de votre site. Jusqu'ici, nous étions parvenus à passer des variables de page en page via la méthode GET (en modifiant l'URL : `page.php?variable=valeur`) et via la méthode POST (à l'aide d'un formulaire).

Mais imaginez maintenant que vous souhaitez transmettre des variables sur toutes les pages de votre site pendant la durée de la présence d'un visiteur. Ce ne serait pas facile avec GET et POST car ils sont plutôt faits pour transmettre les informations une seule fois, d'une page à une autre. On sait ainsi envoyer d'une page à une autre le nom et le prénom du visiteur, mais dès qu'on charge une autre page ces informations sont « oubliées ». C'est pour cela qu'on a inventé les sessions.

Fonctionnement des sessions

Comment sont gérées les sessions en PHP ? Voici les trois étapes à connaître.

- Un visiteur arrive sur votre site. On demande à créer une session pour lui. PHP génère alors un numéro unique. Ce numéro est souvent très gros et écrit en hexadécimal, par exemple : a02bbffc6198e6e0cc2715047bc3766f. Ce numéro sert d'identifiant et est appelé « ID de session » (ou PHPSESSID). PHP transmet automatiquement cet ID de page en page en utilisant généralement un cookie.
- Une fois la session générée, on peut créer une infinité de variables de session pour nos besoins. Par exemple, on peut créer une variable `$_SESSION['nom']` qui contient le nom du visiteur, `$_SESSION['prenom']` qui contient le prénom, etc. Le serveur conserve ces variables même lorsque la page PHP a fini d'être générée. Cela veut dire que, quelle que soit la page de votre site, vous pourrez récupérer par exemple le nom et le prénom du visiteur via la superglobale `$_SESSION` !
- Lorsque le visiteur se déconnecte de votre site, la session est fermée et PHP « oublie » alors toutes les variables de session que vous avez créées. Il est en fait difficile de savoir précisément quand un visiteur quitte votre site. En effet, lorsqu'il ferme son navigateur ou va sur un autre site, le vôtre n'en est pas informé. Soit le visiteur clique sur un bouton « Déconnexion » (que vous aurez créé) avant de s'en aller, soit on attend quelques minutes d'inactivité pour le déconnecter automatiquement : on parle alors de **timeout**. Le plus souvent, le visiteur est déconnecté par un timeout.

Tout ceci peut vous sembler un peu compliqué, mais c'est en fait très simple à utiliser. Vous devez connaître deux fonctions :

- **session_start()** : démarre le système de sessions. Si le visiteur vient d'arriver sur le site, alors un numéro de session est généré pour lui. Vous devez appeler cette fonction au tout début de chacune des pages où vous avez besoin des variables de session.
- **session_destroy()** : ferme la session du visiteur. Cette fonction est automatiquement appelée lorsque le visiteur ne charge plus de page de votre site pendant plusieurs minutes (c'est le timeout), mais vous pouvez aussi créer une page « Déconnexion » si le visiteur souhaite se déconnecter manuellement.

Exemple d'utilisation des sessions

```
<?php
    // On démarre la session AVANT d'écrire du code HTML
    session_start();

    // On s'amuse à créer quelques variables de session dans $_SESSION
    $_SESSION['prenom'] = 'Jean';
    $_SESSION['nom'] = 'Dupont';
    $_SESSION['age'] = 24;
?>
```

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="fr" >
  <head>
    <title>Titre de ma page</title>
    <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
  </head>
  <body>

    <p>
      Salut <?php echo $_SESSION['prenom']; ?> !<br />
      Tu es à l'accueil de mon site (index.php). Tu veux aller sur une autre page ?
    </p>

    <p>
      <a href="mapage.php">Lien vers mapage.php</a><br />
      <a href="monscript.php">Lien vers monscript.php</a><br />
      <a href="informations.php">Lien vers informations.php</a>
    </p>

  </body>
</html>
```

Détruire manuellement une session

Si vous voulez détruire manuellement la session du visiteur, vous pouvez faire un lien « Déconnexion » amenant vers une page qui fait appel à la fonction **session_destroy()**. Néanmoins, sachez que sa session sera automatiquement détruite au bout d'un certain temps d'inactivité.

L'utilité des sessions en pratique

Concrètement, les sessions peuvent servir dans de nombreux cas sur votre site (et pas seulement pour retenir un nom et un prénom !). Voici quelques exemples :

- Imaginez un script qui demande un login et un mot de passe pour qu'un visiteur puisse se « connecter » (s'authentifier). On peut enregistrer ces informations dans des variables de session et se souvenir de l'identifiant du visiteur sur toutes les pages du site !
- Puisqu'on retient son login et que la variable de session n'est créée que s'il a réussi à s'authentifier, on peut l'utiliser pour restreindre certaines pages de notre site à certains visiteurs uniquement. Cela permet de créer toute une zone d'administration sécurisée : si la variable de session login existe, on affiche le contenu, sinon on affiche une erreur.
- On se sert activement des sessions sur les sites de vente en ligne. Cela permet de gérer un « panier » : on retient les produits que commande le client quelle que soit la page où il est. Lorsqu'il valide sa commande, on récupère ces informations et... on le fait payer.

3. Les cookies

Qu'est-ce qu'un cookie ?

Un cookie, c'est un petit fichier que l'on enregistre sur l'ordinateur du visiteur.

Ce fichier contient du texte et permet de « retenir » des informations sur le visiteur. Par exemple, vous inscrivez dans un cookie le pseudo du visiteur, comme ça la prochaine fois qu'il viendra sur votre site, vous pourrez lire son pseudo en allant regarder ce que son cookie contient.

Parfois les cookies ont une mauvaise image. On fait souvent l'erreur de penser que les cookies sont « dangereux ». Non, ce ne sont pas des virus, juste de petits fichiers texte qui permettent de retenir des informations. Au pire, un site marchand peut retenir que vous aimez les appareils photos numériques et vous afficher uniquement des pubs pour des appareils photos, mais c'est tout.

Chaque cookie stocke généralement une information à la fois. Si vous voulez stocker le pseudonyme du visiteur et sa date de naissance, il est donc recommandé de créer deux cookies.

Écrire un cookie

Comme une variable, un cookie a un nom et une valeur. Par exemple, le cookie pseudo aurait chez moi la valeur Matrix.

Pour écrire un cookie, on utilise la fonction PHP **setcookie** (qui signifie « Placer un cookie » en anglais).

On lui donne en général trois paramètres, dans l'ordre suivant :

- le nom du cookie (ex. : pseudo) ;
- la valeur du cookie (ex. : Matrix) ;
- la date d'expiration du cookie, sous forme de timestamp (ex. : 1090521508).

Le paramètre correspondant à la date d'expiration du cookie mérite quelques explications. Il s'agit d'un **timestamp**, c'est-à-dire du nombre de secondes écoulées depuis le 1er janvier 1970. Le **timestamp** est une valeur qui augmente de 1 toutes les secondes. Pour obtenir le timestamp actuel, on fait appel à la fonction **time()**. Pour définir une date d'expiration du cookie, il faut ajouter au « moment actuel » le nombre de secondes au bout duquel il doit expirer.

Si vous voulez supprimer le cookie dans un an, il vous faudra donc écrire : `time() + 365*24*3600`. Cela veut dire : *timestamp actuel* \$+\$ *nombre de secondes dans une année*. Cela aura pour effet de supprimer votre cookie dans exactement un an. Voici donc comment on peut créer un cookie :

```
<?php setcookie('pseudo', 'Matrix', time() + 365*24*3600); ?>
```

Sécuriser son cookie avec le mode httpOnly

Il est recommandé toutefois d'activer l'option httpOnly sur le cookie. Sans rentrer dans les détails, cela rendra votre cookie inaccessible en JavaScript sur tous les navigateurs qui supportent cette option (c'est le cas de tous les navigateurs récents.). Cette option permet de réduire drastiquement les risques de faille XSS sur votre site, au cas où vous auriez oublié d'utiliser htmlspecialchars à un moment.

Il est donc **recommandé** de créer votre cookie plutôt comme ceci :

```
<?php setcookie('pseudo', 'Matrix', time() + 365*24*3600, null, null, false, true); ?>
```

Le dernier paramètre true permet d'activer le mode httpOnly sur le cookie, et donc de le rendre en quelque sorte plus sécurisé. Ça ne coûte rien et vous diminuez le risque qu'un jour l'un de vos visiteurs puisse se faire voler le contenu d'un cookie à cause d'une faille XSS.

Les paramètres du milieu sont des paramètres que nous n'utilisons pas, nous leur avons donc envoyé null.

Créer le cookie avant d'écrire du HTML

Il y a un petit problème avec setcookie... Comme pour session_start, cette fonction ne marche QUE si vous l'appellez avant tout code HTML (donc avant la balise <!DOCTYPE>).

Voyons maintenant comment faire pour écrire deux cookies, un qui retient mon pseudo pendant un an, et un autre qui retient le nom de mon pays :

```
<?php
setcookie('pseudo', 'Matrix', time() + 365*24*3600, null, null, false, true); // On écrit un cookie
setcookie('pays', 'France', time() + 365*24*3600, null, null, false, true); // On écrit un autre
cookie...
```

```
// Et SEULEMENT MAINTENANT, on peut commencer à écrire du code html
?>
```

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="fr" >
  <head>
    <title>Ma super page PHP</title>
    <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
  </head>
  <body>
```

Afficher un cookie

C'est la partie la plus simple. Avant de commencer à travailler sur une page, PHP lit les cookies du client pour récupérer toutes les informations qu'ils contiennent. Ces informations sont placées dans la superglobale `$_COOKIE`, sous forme d'array, comme d'habitude.

De ce fait, si on veut ressortir le pseudo du visiteur qu'on avait inscrit dans un cookie, il suffit d'écrire : `$_COOKIE['pseudo']`.

Ce qui nous donne ce code PHP pour afficher de nouveau le pseudo du visiteur :

<p>

Hé ! Je me souviens de toi !

Tu t'appelles <?php echo \$_COOKIE['pseudo']; ?> et tu viens de <?php echo \$_COOKIE['pays']; ?> c'est bien ça ?

</p>

Modifier un cookie existant

Pour modifier un cookie il faut refaire appel à `setcookie` en gardant le même nom de cookie, ce qui « écrasera » l'ancien.

Par exemple, si j'habite maintenant en Chine, je ferai :

```
setcookie('pays', 'Chine', time() + 365*24*3600, null, null, false, true);
```

Notez qu'alors le temps d'expiration du cookie est remis à zéro pour un an.